

EloidaOS — Current State and the Path to Safety-Critical Systems

Author: Almir Bispo · Technical Document · 2026

Version: 15 tasks + integrated IRQs + Multi-core with truth table

Table of Contents

1. Overview
2. Current Architecture
3. IRQs Integrated into the Model
4. Multi-Core: One System per Core
5. Global Truth Table (Optional)
6. What Is Already Done
7. What Is Missing for Safety-Critical Systems
8. Implementation Roadmap
9. Final Considerations

1. Overview

EloidaOS is a cooperative, deterministic kernel for microcontrollers, founded on a simple mathematical property: **the sum of distinct primes is unique for each combination**. This property allows the complete state of a system of interdependent tasks to be encoded in a single integer.

This document describes the current state of the architecture — with integrated IRQs and multi-core extension — and what remains to be done before the kernel is usable in **safety-critical systems** (DO-178C, ISO 26262, IEC 62304, IEC 61508).

2. Current Architecture

2.1. Founding Principles

- 1. **Time is prime** — coprime intervals, temporal determinism.
- 2. **Task is verdict** — each task returns its prime (OK) or o (failure).
- 3. **MMC is correlation** — a unique sum identifies the state.
- 4. **Dispatch is policy** — an explicit, auditable table.
- 5. **Everything is static** — no heap, no dynamic allocation.
- 6. **Everything is verifiable** — the policy is inspectable line by line.
- 7. **Everything is minimal** — the kernel fits in hundreds of bytes.

2.2. Kernel Structure

```
ISR → IRQ_OPS (flag)
main: IRQ()    → routes flags to IRQ tasks → OK[tid]
main: TASKS()  → periodic tasks → OK[tid]
main: mmc()    → sums all OK[]
main: DESVIO() → selects the task to run
main: runs the arbitrated task
```

2.3. Per-Task Footprint

Element	Size	Purpose
OK[tid]	1 byte	Verdict (prime or o)
DONE[tid]	1 byte	Has run at least once
COUNTS[tid]	4 bytes	Cycle counter
TASK_TABLE[tid]	4 bytes (flash)	Pointer to the task
Total	6 bytes RAM	+ 4 bytes flash

3. IRQs Integrated into the Model

3.1. The Role of the ISR

The hardware ISR is **minimal**: it only sets a flag in `IRQ_OPS` and clears the peripheral flag. If a critical action is required (e.g., shutting down PWM on overcurrent), it is performed inside the ISR itself — but **validation** and the **verdict** remain in the IRQ task.

3.2. The Role of the IRQ Task

The IRQ task is the natural place to:

- Read the raw hardware data;
- Validate (range, CRC, protocol);
- Publish to `SHARED` ;
- Return the verdict (prime or o).

Symmetry: periodic tasks and IRQ tasks share the same contract. One reads a sensor by time; the other reads hardware by event. Both validate, publish, and return prime or o.

3.3. A Single Dispatch Policy

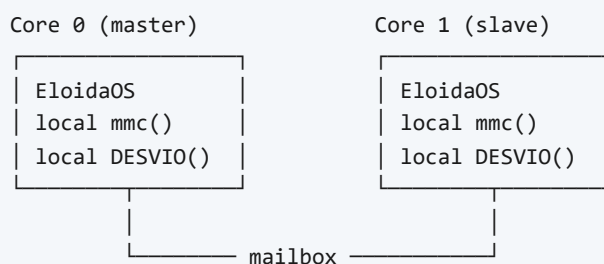
`IRQ_TASK_MAP` is **not a policy dispatch** — it is a **hardware routing table**. It says which task handles which IRQ, and that is a fixed property of the hardware, not of the control policy.

There is therefore **a single policy dispatch**: the `DESVIO(soma)` function, which consults the complete system state.

4. Multi-Core: One System per Core

4.1. The Model

Each core runs a **complete, independent EloidaOS instance**: its own `OK[]` , `IRQ_OPS` , `mmc()` , and `DESVIO()` . Inter-core communication is done through a **hardware mailbox**, treated as **just another peripheral**.



Advantage: multi-core is treated as a peripheral. The kernel does not change. Local determinism is preserved.

4.2. Communication via Mailbox

The mailbox generates an IRQ. The IRQ sets a flag. The flag routes to an IRQ task. The IRQ task reads the message, validates it, and returns a verdict. That verdict enters the local MMC of the receiving core.

5. Global Truth Table (Optional)

5.1. When to Use

The global truth table is used **only when the user wants a global view** of the cores — for example, when the master core must delegate tasks or take over responsibilities from failed cores.

5.2. How It Works

The master maintains `USER_FLAGS` with custom flags that represent the state of remote cores (OK, failure, synchronized, etc.). A `TABELA_GLOBAL` maps combinations of those flags to global actions.

Core state (bitmask) → global action

CORE1_OK	CORE2_OK	CORE3_OK	NORMAL_MODE
CORE2_OK	CORE3_OK		TAKE_MOTOR
CORE1_OK	CORE3_OK		TAKE_LOGGING
CORE3_OK			DEGRADED_MODE
0			FAILSAFE

Important: the truth table is **optional**. Each core remains a pure EloidaOS. Custom flags and the truth table are an **integration mechanism**, not a mandatory layer.

6. What Is Already Done

Item	Status	Notes
Cooperative kernel	DONE	process/run/destroy
MMC by primes	DONE	Unique sum

Table-driven dispatch	DONE	Auditable policy
Integrated IRQs	DONE	Minimal ISR + IRQ task
Validation in IRQ task	DONE	Cohesive and symmetric
Multi-core as peripheral	DONE	Mailbox treated as IRQ
Global truth table	DONE	Optional, master only
Minimal footprint	DONE	6 bytes/task

7. What Is Missing for Safety-Critical Systems

For EloidaOS to be usable in safety-critical systems, protection layers must be added that **do not break the kernel's principles**. The table below summarizes what is missing.

Requirement	Status	How to implement
Interface contract	TODO	<code>TASK_WRITE_MASK[]</code> + <code>_Static_assert</code>
Transactional snapshot	TODO	Copy <code>SHARED</code> before/after critical tasks
Stack canary	TODO	<code>CANARY_VALUE</code> in each critical task frame
Per-task watchdog	TODO	<code>task_deadline[]</code> + hardware timer
PMP / MPU	VERIFY	Test whether CH32V307 implements standard RISC-V PMP
Auditable fault log	PARTIAL	<code>FAULT_LOG[]</code> exists; add timestamp and MMC
Two-domain partition	TODO	Fast domain (ns/us) and critical domain (ms+)
Traceability matrix	TODO	Document: requirement → mechanism → test

7.1. Why PMP Is the Key Piece

True memory protection (spatial partitioning) is only possible with PMP/MPU. On the CH32V307, it must be **empirically verified** whether the QingKe V4F core implements the standard RISC-V PMP or a proprietary mechanism.

If it does, critical tasks (ms+) can be protected with isolated memory regions, leaving fast tasks (ns/us) without PMP to preserve determinism.

7.2. Two-Domain Partition

CRITICAL DOMAIN (PMP + snapshot + watchdog)
- ms tasks (task6+)
- Dedicated stack per task
- Communication via SHARED with contract
FAST DOMAIN (no PMP, no dedicated stack)
- ns/us tasks (task0–task5)
- Shared stack
- Communication via SHARED with contract

8. Implementation Roadmap

Phase 1 — Interface contract **zero cost**

- Define `TASK_READ_MASK[]` and `TASK_WRITE_MASK[]`.
- Add `_Static_assert` to validate at compile time.
- Document the contract as part of the dispatch policy.

Phase 2 — Snapshot + canary **~60 bytes**

- Implement `execute_critical()` for tasks 6+.
- Implement `execute_fast()` for tasks 0–5.
- Add `FaultLog` with timestamp and MMC.

Phase 3 — Per-task watchdog **~10 cycles**

- Add `task_deadline[]` as a const table.
- Implement detection in `SysTick_Handler`.
- Add `longjmp` for real deadline enforcement.

Phase 4 — PMP **~0.17 μ s/critical task**

- Empirically test PMP on the CH32V307.
- Configure PMP regions for each critical task.
- Implement `switch_to_user_mode` / `switch_to_machine_mode` .
- Capture invalid-access exceptions.

Phase 5 — Certification evidence

- Generate fault logs to flash.
- Document the dispatch table as the control policy.
- Produce a traceability matrix (requirement → mechanism → test).
- Create hardware-free tests (flag injection + assert).

9. Final Considerations

ElroidaOS is currently in a **cohesive and lean** state:

- A single policy dispatch (`DESVIO`).
- A hardware routing table (`IRQ_TASK_MAP`).
- IRQ tasks with the same contract as periodic tasks.
- Multi-core as a peripheral, with an optional truth table on the master.
- A footprint of 6 bytes per task.

For safety-critical systems, **protection layers** must be added that respect the kernel's principles:

- Interface contract (zero cost).
- Transactional snapshot (~60 bytes).
- Stack canary (~8 bytes).
- Per-task watchdog (~10 cycles).
- PMP for critical tasks (~0.17 μ s/task).

The key is to **partition into two domains**:

- **Fast domain** (ns/us): no PMP, with contract + canary.
- **Critical domain** (ms+): PMP + snapshot + watchdog.

Conclusion: EloidaOS turns an ordinary MCU into a deterministic controller of interdependent events, with minimal memory cost and full verifiability of the control policy. With the appropriate protection layers, it becomes suitable for safety-critical systems — preserving the determinism, verifiability, and minimal footprint that define it.

EloidaOS — Current State and the Path to Safety-Critical Systems

Author: Almir Bispo · 2026